

MODULE 4

Basic Web Concepts & Security Risks

Module Overview

This module introduces students to the fundamental technologies that underpin the World Wide Web and, critically, how attackers exploit them. Before any meaningful web security work can be done, students must understand what happens behind the scenes every time a browser loads a page from DNS resolution and HTTP requests to server responses and browser rendering.

Rather than focusing exclusively on attack techniques, this module builds a solid conceptual foundation of how the web works, then maps common security risks onto that foundation. This dual understanding of normal behavior alongside attack behavior is essential for every subsequent module covering web vulnerability testing, OWASP, and ethical hacking. Students who complete this module will never look at a URL, a login form, or an error message the same way again.

Learning Objectives

By the end of this module, students will be able to:

- Explain how a web browser communicates with a web server using HTTP and HTTPS
- Describe the structure of a URL and identify the security-relevant components of each part
- Explain the role of DNS in web communication and describe how DNS can be abused
- Differentiate between client-side and server-side processing and explain the security implications of each
- Describe the HTTP request-response cycle, including common methods (GET, POST, PUT, DELETE) and status codes
- Explain what cookies, sessions, and authentication tokens are and why they are high-value targets
- Identify common web security risks including the most frequently exploited categories
- Describe how HTTPS and TLS protect web traffic — and what they do not protect
- Explain the browser's Same-Origin Policy and why it matters for web security
- Analyse a described web interaction and identify where security controls exist or are absent

Why Web Security Matters

The web is the largest attack surface in the history of computing. Every internet-connected organization operates web applications, customer portals, internal dashboards, APIs, login systems, and e-commerce platforms and each of these is a potential entry point for attackers.

Web application attacks consistently rank as one of the most common initial access vectors in security breaches. Understanding the web is not optional for cybersecurity professionals; it is foundational, regardless of which specialization they eventually pursue.

Web security matters because it protects websites and users from attacks, data theft, and unauthorized access, keeping information safe and services reliable.

The Security Professional's Perspective

Web vulnerabilities appear in penetration testing, incident response, malware analysis, threat intelligence, and even physical security assessments.

The OWASP Top 10 the de facto standard list of the most critical web application security risks is referenced in security certifications, compliance frameworks, and job descriptions worldwide. A security analyst who does not understand how the web works cannot meaningfully defend, test, or investigate it.

In security, **de facto** means a tool, standard, or practice that is **widely used and trusted in real-world work**, even if it's not officially enforced by law or a formal rule.

Three Reasons Every Security Professional Must Understand the Web:

- **Ubiquity:** Web applications are everywhere and almost every breach involves one at some point in the attack chain
- **Complexity:** The web stack involves multiple interacting technologies, each with its own vulnerabilities and trust assumptions
- **Attacker focus:** In web security, attackers mainly focus on **web applications and their weaknesses**, such as login pages, input fields, APIs, sessions, and databases, because these are the easiest entry points to steal data, take control, or break systems.

How the Web Works - The Request-Response Cycle

Every web interaction from loading a news article to submitting a bank transfer follows the same fundamental pattern: a client (browser) sends a request to a server, and the server returns a response. Understanding this cycle in detail is the foundation of all web security work.

The **request-response cycle** is how the web works: a client (like your browser) sends a request to a server, and the server replies with a response (like a webpage or data).

Step-by-Step: What Happens When You Visit a Website

Step	What Happens & Security Relevance
1. DNS Resolution	The browser queries a DNS server to translate the domain name (e.g. <code>www.google.com</code>) into an IP address. DNS is unencrypted by default and responses can be intercepted or forged (DNS spoofing/poisoning).
2. TCP Connection	The browser establishes a TCP connection to the server on port 80 (HTTP) or port 443 (HTTPS) using the three-way handshake. Port 443 initiates a TLS handshake immediately after.
3. TLS Handshake (HTTPS)	For HTTPS, the server presents a digital certificate. The browser validates it against trusted Certificate Authorities. If valid, an encrypted channel is established. Attackers cannot read this traffic but can still manipulate the application layer.

4. HTTP Request Sent	The browser sends an HTTP request including the method (GET/POST), the path, headers (cookies, authentication tokens, User-Agent), and optionally a body (for POST/PUT requests containing form data).
5. Server Processes Request	The web server (and any backend application) processes the request querying databases, running business logic, and generating a response. Injection attacks target this step.
6. HTTP Response Returned	The server returns a status code, headers (including security headers like Content-Security-Policy), and the response body (HTML, JSON, file, etc.).
7. Browser Renders Response	The browser parses HTML, executes JavaScript, loads CSS and images. XSS attacks target this final step injecting malicious scripts that execute in the victim's browser.

Anatomy of a URL

A URL (Uniform Resource Locator) is far more than just a web address it is a structured string that contains multiple security-relevant components. Security professionals must be able to parse every component at a glance.

```
https://login.example.com:443/account/reset?token=abc123&user=admin#form
```

Component	Example Value	Security Significance
Scheme / Protocol	https://	Determines whether communication is encrypted (HTTPS) or plaintext (HTTP). Always verify HTTPS before submitting credentials.
Subdomain	login.	Subdomains can be taken over if DNS entries point to expired cloud resources (subdomain takeover). Attackers register lookalike subdomains to phish users.
Domain	example.com	The registered domain. Attackers use typosquatting (example.com, example-secure.com) to deceive users. Always check the root domain carefully.
Port	:443	Port 443 is standard for HTTPS; port 80 for HTTP. Unusual ports (e.g. :8080, :3000) may indicate development servers with weaker security controls.
Path	/account/reset	Defines the resource requested. Security misconfigurations can expose sensitive paths (e.g. /admin, /backup, /.git) to unauthorised users.

Query String	?token=abc123&user=admin	Parameters passed to the server. Sensitive data (tokens, IDs) should never appear in query strings they appear in browser history, server logs, and referrer headers.
Fragment	#form	Processed entirely by the browser; never sent to the server. Cannot be logged server-side sometimes misused in client-side redirect attacks.

HTTP - Methods, Headers & Status Codes

HTTP Methods

HTTP defines a set of request methods that indicate the desired action. Understanding which method is used for which action is critical for identifying insecure API designs and access control weaknesses.

Method	Purpose	Security Notes
GET	Retrieve a resource (read-only)	Parameters passed in URL - never use GET to submit passwords or sensitive data. GET requests are logged in full by servers, proxies, and browser history.
POST	Submit data to create or process a resource	Data sent in request body - not visible in URL, but still transmittable in plaintext over HTTP. Always use HTTPS. POST is the correct method for login forms and file uploads.
PUT	Replace an entire resource at a specified location	If not properly access-controlled, PUT can allow attackers to overwrite files or data. Often found in REST APIs - must require authentication.
DELETE	Remove a specified resource	Dangerous if unauthenticated - an attacker could delete records, user accounts, or configurations. Must require strong authorization checks.
PATCH	Partially modify an existing resource	Similar risks to PUT but affects partial data. Mass assignment vulnerabilities can occur if update fields are not whitelisted.
OPTIONS	Query which HTTP methods a server supports	Can reveal what actions are available - attackers use this for reconnaissance. CORS misconfigurations often discovered via OPTIONS requests.
HEAD	Same as GET but returns headers only (no body)	Can be used to check if a resource exists without downloading it - useful for reconnaissance to confirm sensitive paths without triggering full responses.

Important HTTP Status Codes

Status codes communicate the outcome of a request. Security professionals read status codes constantly during testing and log analysis they reveal information about application behavior and misconfigurations.

Code	Meaning	Security Relevance
200 OK	Request succeeded	Normal response - but content must still be inspected for sensitive data exposure.
301/302	Redirect	Open redirect vulnerabilities can redirect users to attacker-controlled sites. Check that redirect destinations are validated.
400 Bad Request	Malformed request syntax	Can reveal information about expected input format - useful to attackers during probing.
401 Unauthorized	Authentication required	Correct response for unauthenticated access. If missing, an application may be serving protected resources without checking identity.
403 Forbidden	Authenticated but access denied	Confirms the resource exists - useful reconnaissance for attackers even though access is denied.
404 Not Found	Resource does not exist	Differentiating 403 vs 404 reveals whether a path exists. Some applications return 404 for both to prevent path enumeration.
500 Internal Server Error	Unexpected server failure	May expose stack traces, technology versions, database errors - valuable information for attackers. Error messages should never be exposed to end users.
503 Service Unavailable	Server temporarily unavailable	Could indicate a DoS attack is succeeding. Monitor for unexpected 503 spikes in logs.

Cookies, Sessions & Authentication Tokens

HTTP is a stateless protocol - every request is independent and the server has no memory of previous requests. To maintain state (e.g. keeping a user logged in), web applications use cookies, server-side sessions, and tokens. These mechanisms are among the most attacked components of any web application.

How Session Management Works

Mechanism	How It Works & Security Considerations
Cookies	Small pieces of data stored by the browser and sent automatically with every request to the matching domain. Can hold session IDs, preferences, or authentication tokens. Critical attributes: HttpOnly (no JS access), Secure (HTTPS only), SameSite (CSRF protection).
Server-Side Sessions	The server generates a unique session ID and stores session data server-side (database or memory). The session ID is sent to the client as a cookie. The

	actual data never leaves the server — only the ID does. Secure if session IDs are sufficiently random and expire properly.
JWT (JSON Web Tokens)	A signed token containing user claims (e.g. user ID, role) stored client-side and sent with requests. The server validates the signature without storing state. Risks: weak signing keys, algorithm confusion attacks (changing alg: HS256 to none), and sensitive data in the payload (which is only base64 encoded, not encrypted by default).
OAuth 2.0 / OIDC	Delegation frameworks that allow users to authorise third-party applications without sharing passwords. Used for 'Login with Google/GitHub/Microsoft.' Misconfigurations in redirect_uri validation or state parameter handling can lead to account takeover.

Common Session & Authentication Attacks:

- **Session Hijacking:** Stealing a valid session cookie (via XSS, network sniffing, or access to logs) to impersonate an authenticated user
- **Session Fixation:** Forcing a user to authenticate with a known session ID so the attacker can immediately access the session
- **Brute Force / Credential Stuffing:** Attempting thousands of username/password combinations; using breached credentials to log into other services
- **JWT Algorithm Confusion:** Exploiting poorly implemented JWT libraries to forge tokens by switching to a weak or unsigned algorithm
- **CSRF (Cross-Site Request Forgery)** Tricking an authenticated user's browser into making an unintended request abusing the automatic cookie attachment behaviour

Client-Side vs Server-Side Processing

One of the most fundamental distinctions in web security is understanding what runs in the browser (client-side) versus what runs on the server (server-side). This distinction determines what an attacker can see, modify, and exploit.

Aspect	Client-Side	Server-Side
Location	Executes in the user's browser	Executes on the web/application server
Language	HTML, CSS, JavaScript	PHP, Python, Node.js, Ruby, Java, .NET
Visibility	All source code visible to the user (View Source, DevTools)	Source code is hidden from users - only output is returned
Attacker control	Attacker can modify client-side code, variables, and requests in browser DevTools	Attacker cannot directly modify server code - must manipulate inputs
Key security risk	XSS - injecting malicious scripts that execute in other users' browsers	SQL Injection, Command Injection - injecting malicious input that the server executes

Trust level	Never trust - all client-side validation can be bypassed	Should be the authoritative layer for all security decisions
Data storage	localStorage, sessionStorage, cookies (accessible to JS unless HttpOnly)	Databases, server memory - not directly accessible to clients

Critical Rule: Never rely solely on client-side validation for security. Anything that runs in the browser can be bypassed. An attacker with browser DevTools can disable JavaScript, modify form values, remove input restrictions, and replay modified requests all in seconds. All security-critical validation must be enforced server-side.

Common Web Security Risks

This section introduces the most frequently encountered web security vulnerabilities. Each will be covered in greater depth in Module 17 (Web Security Fundamentals & OWASP Top 10). The goal here is to establish conceptual awareness before the hands-on work begins.

Injection Attacks

Injection attacks occur when an attacker sends untrusted data to an interpreter a database engine, operating system shell, or XML parser as part of a command or query. The interpreter cannot distinguish between legitimate instructions and attacker-supplied data.

Type	How It Works	Example Impact
SQL Injection	Malicious SQL is inserted into an input field and executed by the database	Bypass authentication, extract entire databases, modify or delete records
Command Injection	OS commands are injected through application input and executed by the server	Execute arbitrary commands on the server, gain a shell, install backdoors
LDAP Injection	Manipulating LDAP queries used for directory-based authentication	Bypass authentication, enumerate directory users and groups
XML/XPath Injection	Injecting malicious XML or XPath expressions into XML-processing functions	Extract data from XML documents, bypass authentication in XML-based systems

Cross-Site Scripting (XSS)

XSS is one of the most prevalent web vulnerabilities. It occurs when an application includes untrusted data in a web page without proper validation or escaping, allowing attackers to execute scripts in the victim's browser.

XSS Type	Mechanism	Impact
Stored (Persistent)	Malicious script is saved to the database and served to all users who view the affected page	Mass cookie theft, defacement, redirecting all visitors to malicious sites
Reflected	Malicious script is included in the URL and reflected back in the response — victim must click a crafted link	Session theft, credential harvesting, fake login page injection
DOM-Based	The vulnerability exists in client-side JavaScript that reads attacker-controlled data from the DOM without sanitisation	Bypasses server-side filtering — entirely client-side exploitation

Other Key Web Vulnerabilities

Vulnerability	Description & Impact
Broken Access Control	The most common OWASP Top 10 category. Users can access resources or perform actions beyond their intended permissions e.g. accessing another user's account data by changing an ID in the URL (IDOR - Insecure Direct Object Reference).
Security Misconfiguration	Default credentials left unchanged, unnecessary features enabled, verbose error messages, open cloud storage buckets, missing security headers. Extremely common and often the easiest vulnerability to exploit.
Insecure Direct Object Reference (IDOR)	A type of broken access control - directly referencing an internal implementation object (e.g. /invoice?id=1042) without verifying the requesting user has permission to access that specific record.
Open Redirect	The application accepts a user-controlled URL parameter and redirects to it without validation. Used in phishing attacks to make malicious URLs appear to originate from a trusted domain.
Clickjacking	The attacker embeds the target site in a transparent iframe and overlays a fake UI - tricking users into clicking elements on the real site unknowingly. Prevented by X-Frame-Options and CSP frame-ancestors.
File Upload Vulnerabilities	Allowing users to upload files without validating type, content, or storage location can enable uploading of web shells (malicious scripts executed by the server) or path traversal to overwrite system files.